CODE SUMMARIZATION

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Specify the Summary Intent Before Asking

- State what kind of summary you want: what the code does, why it exists, how to use it, its properties, or how it is implemented. Different audiences and tasks need different summary types. Without a pinned intent, the AI defaults to a vague 'what it does' blurb.

Reasoning

- Five distinct summary intent categories (what, why, how-it-is-done, property, how-to-use) serve different comprehension needs in real projects.
- LLMs produce high-quality summaries across all five categories only when intent is explicit; unconditioned prompts default to what summaries and miss what the reader actually needs.
- Real-world Java projects use these categories at different rates; a what summary often omits rationale, preconditions, or usage examples that matter most for a given reader.

Key Practices: Prompting for a Specific Intent

- Name the intent explicitly (e.g., 'give me a why summary explaining the design rationale' or 'give me a how-to-use summary with a short call example').
- Tell the model who the reader is (maintainer, caller, reviewer), because that determines which intent is most useful.
- Ask for one intent at a time unless you genuinely want a multi-part summary; blended prompts tend to be long and shallow.

Key Practices: Reviewing the Result

- Check that the output actually matches the intent you requested, not just any true statement about the code.
- Reject summaries that restate syntax line by line when you asked for rationale or usage.
- If you need multiple intents (e.g., a Javadoc header), request them as separate labelled sections so you can evaluate each one independently.

Guideline 2: Surface the Semantic Facts the AI Cannot See

- When summarizing a function, attach the semantic facts a quick read would extract: repository name, fully qualified function name, identifier names, data-flow relationships, and pre- and post-conditions. The AI will often miss these even though they look obvious from the code.

Reasoning

- Adding repo name, function name, identifiers, and data-flow graph to the prompt improved summary quality across three LLMs on standard benchmarks.
- Explicitly labelled semantic facts focus the model's attention on names and flows that carry intent, benefiting even capable models.

Key Practices: Prompting with Semantic Facts

- Include the repository or module name and the fully qualified function name so the model can ground its vocabulary in the project.
- List non-trivial identifiers (parameters, key local variables, called functions) with a short note if their meaning is not obvious from the name.
- State any pre- or post-conditions and the main data-flow relationship (what flows from inputs into the return value) in one or two lines.

Key Practices: Reviewing the Result

- Check that project-specific terms from the facts appear in the summary rather than generic synonyms.
- Flag summaries that invent behaviour not supported by the facts you provided.
- If the summary repeats the facts verbatim instead of synthesising them, tighten the prompt to ask for a summary, not a restatement.

Guideline 3: Verify the Summary Against Behaviour, Not Just Plausibility

- Treat an AI-generated summary as a claim about the code's behaviour and check it. Pick one or two testable statements and confirm them by running the code or writing a quick unit test. Do not accept a summary because it sounds right.

Reasoning

- Manual inspection of 540 LLM-generated comments found demonstrably inaccurate statements in roughly one fifth of cases, even for the strongest model.
- Existing code-comment consistency detectors had no statistically significant relationship with actual accuracy, so surface checks cannot catch these errors.
- 'Document testing' turns each summary claim into an executable test -- directly applicable without specialised tooling.

Key Practices: Prompting for a Verifiable Summary

- Ask the model to produce a summary whose claims are phrased as concrete behaviours (inputs, outputs, side effects, error cases) rather than vague prose.
- Ask it to list the summary's claims as a short bullet list at the end, so each one can be checked independently.
- When the summary hinges on an edge case (empty input, negative number, concurrent access), ask the model to state the expected behaviour explicitly.

Key Practices: Reviewing the Result

- Pick the most load-bearing claim and confirm it by running the function on a matching input.
- Write a one-line assertion or REPL expression for any claim about return values, exceptions, or ordering.
- Treat unverified claims about thread safety, performance, or side effects with extra suspicion as these are the classes where LLM-generated comments most often drift.

References (1 of 2)

- [1] L. Pascarella and A. Bacchelli. "Classifying Code Comments in Java Open-Source Software Systems." 14th International Conference on Mining Software Repositories (MSR), 2017. DOI: 10.1109/MSR.2017.63.
- [2] M. Geng et al. "Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning." ICSE 2024, pp. 453-465. arXiv:2304.11384. DOI: 10.1145/3597503.3608134.
- [3] T. Ahmed et al. "Automatic Semantic Augmentation of Language Model Prompts (for Code Summarization)." ICSE 2024. arXiv:2304.06815. DOI: 10.1145/3597503.3639183.
- [4] G. Sridhara et al. "Icing on the Cake: Automatic Code Summarization at Ericsson." ICSME Industry Track, 2024. arXiv:2408.09735.
- [5] S. Kang et al. "Identifying Inaccurate Descriptions in LLM-generated Code Comments via Test Execution." 2024. arXiv:2406.14836.

References (2 of 2)

- [6] S. Stapleton et al. "A Human Study of Comprehension and Code Summarization." ICPC 2020, pp. 2-13. DOI: 10.1145/3387904.3389258.
- [7] Y. Virk et al. "Calibration of Large Language Models on Code Summarization." Proc. ACM Softw. Eng., Vol. 2, FSE 2025. arXiv:2404.19318. DOI: 10.1145/3729400.
- [8] W. Sun et al. "Source Code Summarization in the Era of Large Language Models." ICSE 2025. arXiv:2407.07959.
- [9] T. Ahmed and P. Devanbu. "Few-shot Training LLMs for Project-Specific Code-Summarization." ASE NIER Track, 2022. arXiv:2207.04237. DOI: 10.1145/3551349.3559555.